

P4197R0

Design questions for trivial relocation in C++29

Published Proposal, 2026-04-19

Author:

[Giuseppe D'Angelo](#)

Audience:

EWG, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Trivial relocation was removed from C++26 due to a number of fundamental design disagreements. This paper does not propose a specific design; instead, it identifies and organizes the open design questions that any future relocation proposal for C++29 must address. The goal is to build consensus on the design space before a concrete proposal is put forward.

Table of Contents

- 1 Changelog**
- 2 Introduction**
 - 2.1 What this paper is and is not
 - 2.2 Prior work
- 3 The semantics of trivial relocation**
 - 3.1 Question 1: Mechanism: byte copying or dedicated operations?
 - 3.2 Question 2: Must trivial relocation always be bitwise?

3.3 Question 3: Relocation and constant evaluation

4 Relocation and the type system

4.1 Question 4: Optimization of move+destroy, or its own primitive?

4.2 Question 5: User control: traits, keywords, or compiler determination?

4.2.1 Opt-in syntax

4.2.2 Conditional opt-in

4.2.3 Interaction with the Rule of Five

4.3 Question 6: Eligibility when subobjects are not trivially relocatable

5 The scope of a relocation proposal

5.1 Question 7: Should relocation work on automatic variables?

5.2 Question 8: Must non-trivial relocation be included?

5.3 Question 9: Must a trivial relocation proposal provide complete library support?

6 Summary of design questions

7 Acknowledgements

References

Non-Normative References

§ 1. Changelog

- R0
 - First submission

§ 2. Introduction

Trivial relocation has been a long-standing goal for C++. Multiple proposals have been put forward over the years, most notably [\[P1144R12\]](#) and [\[P2786R13\]](#), each with a different design and different tradeoffs. In the C++26 cycle, [\[P2786R13\]](#) was adopted in Hagenberg, but was ultimately removed from the C++26 Working Draft in Kona due to a number of unresolved design concerns.

We believe that the failure to reach consensus was not merely a matter of details, but the result of **fundamental disagreements on several core aspects** of what a relocation feature should look like. These

disagreements are not new; they have been present throughout the history of the competing proposals, but were not always surfaced and discussed explicitly.

This paper attempts to catalog these disagreements as a series of design questions. **We do not propose answers**; our goal is to lay out the design space so that future proposals can be evaluated against explicit, agreed-upon criteria.

§ 2.1. What this paper is and is not

This paper is a survey of design questions. It is not a proposal, nor an endorsement of any specific design. It does not contain wording.

We hope that by reaching consensus on these questions first, the Committee can avoid another cycle of last-minute disagreements when a concrete relocation proposal is brought forward for C++29.

§ 2.2. Prior work

The two main competing proposals for trivial relocation have been [\[P1144R12\]](#) ("std::is_trivially_relocatable") and [\[P2786R13\]](#) ("Trivial Relocatability For C++26").

[\[P3233R0\]](#) ("Issues with P2786 (Trivial Relocatability For C++26)") highlighted some issues with the design of an earlier version of P2786. [\[P3937R0\]](#) ("Type Erasure Requirements For Future Trivial Relocation Design") lays down a series of design requirements in order to allow trivial relocation to interact with type erasure facilities.

Late in the C++26 cycle [\[P3858R0\]](#) ("A Lifetime-Management Primitive for Trivially Relocatable Types") proposed a new lower-level primitive for trivial relocation, by splitting the copy aspect of it from the lifetime restart aspect.

On the library side, [\[P3516R2\]](#) ("Uninitialized algorithms for relocation") proposed relocation algorithms; [\[P3780R0\]](#) ("Detecting bitwise trivially relocatable types") proposed the `is_bitwise_trivially_relocatable` trait; [\[P4102R0\]](#) ("Container insertion and erasure should be allowed to relocate") (and [\[P2959R0\]](#) and [\[P3055R1\]](#) before it) propose changes to container operations to allow relocation.

§ 3. The semantics of trivial relocation

The most fundamental questions concern what trivial relocation *means* in the abstract machine.

§ 3.1. Question 1: Mechanism: byte copying or dedicated operations?

Does trivial relocation happen by "just moving bytes in memory" (for instance by using `memcpy`, `memmove`, `realloc`; or equivalent facilities), or does it require dedicated entry points in the abstract machine?

The question here is whether the Standard needs to introduce a new operation in the abstract machine to express trivial relocation, or whether the feature can be realized entirely in terms of the byte-level copying facilities that already exist: `memcpy`, `memmove`, `realloc`, as well as other IPC/serialization mechanisms.

The case for a dedicated entry point builds on a visible trend in recent revisions of the Standard. C++20 added the implicit object creation rules in [intro.object], and C++23 added `std::start_lifetime_as`: both took patterns that had long been "undefined behavior that works in practice" and gave them formal meaning in the abstract machine. A dedicated trivial relocation operation would be a continuation of this direction, taking the `memmove`-based idioms used in third-party container implementations and giving them proper semantics, rather than leaving them as UB that happens to work.

It also bears on questions that cannot be answered purely in terms of bytes: constant evaluation is the clearest example, since byte-level copying is not available at compile time, and a `constexpr` trivial relocation feature might need something defined at a higher level of abstraction (assuming relocation is not defined as a "move+destroy" combination; see § 3.3 [Question 3: Relocation and constant evaluation](#)).

We note in passing that, even among those who agree on the need for a dedicated operation, there has been significant disagreement on its shape: [P2786R13] introduced one such operation, and [P3858R0] argued that a lower-level primitive was more appropriate. The shape of the operation is not the subject of this question, but the difficulty of agreeing on it is worth recording.

The case for blessing "pure" byte copying rests on over 20 years of library practice. Libraries such as Qt, folly, and BSL have all relied on `memmove`, `realloc`, and related byte-level facilities to implement something equivalent to trivial relocation (in the absence of any Standard-provided primitive). [P1144R11] and [P3559R0] contain extensive surveys of such libraries.

The concrete stake for these libraries is whether a standardized primitive covers their existing use cases. Qt, for example, uses `realloc` to grow its container buffers; if the standardized operation does not permit this, Qt will continue to rely on its own mechanism and ignore the Standard feature. The variety of byte-level idioms in real-world use (`memmove`, `memcpy`, `realloc`, `cudaMemcpy`, and others) is broad enough that no fixed list of blessed functions is likely to cover all of them. If we go down

this path, we should likely bless "copying bytes, by whatever means" rather than a specific named operation.

§ 3.2. Question 2: Must trivial relocation always be bitwise?

Is trivial relocation always a bitwise operation, or is there the need (or the will) for a non-bitwise version?

The question here is whether the bitwise representation of every trivially relocatable type is position-independent: can trivial relocation always be performed by a pure bitwise copy, or do some types require a fixup step after the copy?

The concrete case is pointer authentication (see [\[P3751R1\]](#)). Summarizing, on architectures that use it (notably, Apple's arm64e ABI), certain pointer values carry a cryptographic signature derived in part from the storage address of the pointer itself.

The vtable pointer of an object of polymorphic type is one such pointer: when the object is moved to a new address, the old signature is no longer valid, and the pointer must be re-signed against the new address. This fixup cannot be expressed in terms of byte copies alone; it requires an operation that knows that a particular set of bytes is a signed pointer and knows how to re-sign it.

One position is that trivial relocation is always a pure bitwise copy. Under this position, whether a given type is trivially relocatable may depend on the target architecture/ABI: polymorphic types may be trivially relocatable on most widely-deployed targets (e.g. x86_64 with the Itanium C++ ABI), but not on arm64e. The Standard guarantees that, when the trait holds, a bitwise copy is always correct; anything else is simply not trivial relocation at all. The mechanism by which the bitwise trivial relocation is performed, that is, whether through byte-copy facilities alone or through a dedicated language entry point that must be called, is a Question 1 ([§ 3.1 Question 1: Mechanism: byte copying or dedicated operations?](#)) concern.

The other position is that trivial relocation may be non-bitwise. Under this position, polymorphic types are trivially relocatable everywhere, but trivial relocation no longer reduces to a bitwise copy: on architectures like arm64e, some bytes must be fixed up after the copy. How exactly the fixup is performed is again a Question 1 ([§ 3.1 Question 1: Mechanism: byte copying or dedicated operations?](#)) concern, with two main sub-options. In the first, a mandatory language entry point must be called on every trivial relocation; the entry point is a no-op on most platforms but performs the fixup where needed, and user code does not need to know in advance whether a given type requires it. This is the shape of [\[P2786R13\]](#), whose `std::trivially_relocate` is a single operation that both copies bytes and performs the fixup, and also of [\[P3858R0\]](#), which decouples the byte-copy step (performed

by any byte-copy mechanism -- `memcpy`, `realloc`, and so on) from the lifetime-restart call `std::restart_lifetime<T>` that performs the fixup at the destination. In the second sub-option, byte copies alone suffice for most types, and a dedicated fixup primitive must be called *only for types that need it*; this requires a type trait so that user code can tell the two cases apart and dispatch correctly.

The second position (non-bitwise trivial relocation) has cascading consequences elsewhere in the language. A union such as `union U { char c; Poly p; }`; is one such case: the correct fixup depends on which member is active, and that is not known statically. [P2786R13] left the trivial relocation of such unions implementation-defined.

A second consequence concerns a long-standing C++ pattern: storing an object of some type `T` inside a byte buffer owned by a wrapper type `W`. This is the mechanism behind small object optimization, type erasure (for instance `std::function`, `std::any`, or `std::inplace_function`), as well as other facilities. When `W` is bitwise copied, so are the bytes making up the contained `T`, and the library wants `W` to be trivially relocatable whenever `T` is. But under the second position, `T` may be trivially relocatable while requiring a fixup, in which case a bitwise copy of `W`'s buffer is incorrect: `W`'s copy does not know to fix up the `T` it contains (because it has been type erased). The only general solution is to introduce a second trait, "bitwise trivially relocatable", so that `W` can gate its own trivial relocation on `is_bitwise_trivially_relocatable<T>` rather than on `is_trivially_relocatable<T>`. [P2786R13] did not provide such a trait, therefore [P3780R0] proposed it for C++29. [P3937R0] lays out the broader set of requirements trivial relocation must meet to interact correctly with type erasure.

§ 3.3. Question 3: Relocation and constant evaluation

Does trivial relocation need to work during constant evaluation?

The shape of this question depends on the answer to Question 4 (§ 4.1 Question 4: Optimization of move+destroy, or its own primitive?). Under the first framing, where trivial relocation is an optimization of a move followed by a destroy, the operation is already usable during constant evaluation: at compile time the compiler simply does not apply the optimization, and the move-plus-destroy sequence runs as normal. There is no live question to resolve, beyond whether the opt-in machinery itself is `constexpr`-friendly. Under the second framing, where trivial relocation is its own primitive and relocate-only types are possible, the question is substantive: what happens when a user tries to relocate such a type at compile time?

In that case, three positions are possible. One is that trivial relocation must work during constant evaluation as part of the initial feature; the language guarantee is then uniform across evaluation contexts.

The second is that it need not, either temporarily or permanently; trivial relocation is a runtime-only operation, and types whose relocation is required at compile time must use some other mechanism. The third is a compromise: constant-evaluation support is a goal, but if it proves intractable for the first implementations it may be deferred and added later. The remainder of this section catalogs the concrete consequences that bear on this choice.

Without `constexpr` support, relocate-only types cannot be used at compile time: there is no way to move them to a new location. The practical impact of this gap in C++26 was limited, because other pieces of the design were also missing. In particular, relocation of automatic-storage variables was not supported at all, and the container requirements had not been updated to accommodate relocate-only types, so a relocate-only type could not be passed by value, returned by value, or stored in a standard container. The semantic gap remains, however, and would become observable as those other pieces are filled in.

A second consequence concerns the `is_nothrow_relocatable` trait. Some types are throw-movable but nothrow-trivially-relocatable: for instance, `std::list` in MS-STL's implementation, because its move constructor must perform an allocation (and may therefore throw) while its relocation operation is a bitwise copy (and cannot). If `is_nothrow_relocatable<std::list<int>>` reports `true`, it reflects the runtime behavior accurately; but at compile time, without `constexpr` support for trivial relocation, the operation must fall back to the throwing move-plus-destroy path, and the trait's promise no longer holds. To our knowledge this would be the first case in the Standard of a type trait whose promise differs between constant and non-constant evaluation. The issue surfaced during the design of `relocate_at` in [P3516R2].

Finally, a `constexpr` trivial relocation operation cannot be realized in terms of byte-level copying, because byte-level copying is not available during constant evaluation (see §3.1 Question 1: [Mechanism: byte copying or dedicated operations?](#)). How much of a problem this is depends on the shape of the primitive. If trivial relocation is a single all-in-one operation, as in [P2786R13]'s `std::trivially_relocate`, the challenge is entirely on the implementation: the compiler can give the operation a direct compile-time semantics without ever exposing byte copying to the user. If instead the primitive is split into a user-performed byte copy followed by a lifetime-restart call, as in [P3858R0], the design itself must answer a further question: by what mechanism does the user perform the byte copy at compile time, when `memcpy` and friends are unavailable? That question must be resolved before `constexpr` support can be meaningfully claimed under a split-primitive design.

§ 4. Relocation and the type system

How does relocation interact with the existing type properties and type classification in C++?

§ 4.1. Question 4: Optimization of move+destroy, or its own primitive?

Does trivial relocation merely add an optimization of the move+destroy sequence, or does it introduce an entirely new primitive operation?

The question here is what we mean by "relocation" in the first place. Trivial relocation is easy to describe informally: an object is transferred from one place in memory to another, with the source lifetime ending and the destination lifetime beginning. However, the formal account of that operation drives everything else in this question. If relocation is defined as the sequence "move-construct-then-destroy", then trivial relocation is an optimization of that sequence; if instead relocation is its own abstract operation, then trivial relocation is the trivial case of that operation. The two framings lead to different eligibility rules, different interactions with the rest of the language, and different expectations about what future work a relocation feature opens the door to.

Under the first framing, relocation is move construction at the destination followed by destruction at the source. A type is trivially relocatable when that sequence can be replaced by a byte copy (possibly with a [Question 2 \(§ 3.2 Question 2: Must trivial relocation always be bitwise?\)](#)-style fixup). The trait reports a property of a composed operation; it does not introduce a new one.

Under the second framing, relocation is its own abstract operation, with its own semantics: end the object's lifetime at the source, start a new object's lifetime at the destination, and "somehow" transfer the representation in between. A type is trivially relocatable when that operation, on that type, reduces to a byte copy (possibly with a fixup; in any case, no user-provided code is run). The move+destroy sequence may coincide with this operation on most types, but it is not the definition. There is no general relocation primitive in C++ today, and the attempts to add one have stalled (see [\[P2839R0\]](#) and [\[P2785R3\]](#)). Under this framing, the general case can remain deferred while the trivial case is specified on its own.

The existing-practice argument supports the first framing. Every deployed use of trivial relocation today (as found in third-party libraries) is in effect an optimization of move+destroy. The canonical sites are contiguous-container operations (reallocation, insert, erase) in library-specific container types such as [QVector](#). To the best of our knowledge, no deployed library has a notion of a relocate-only type, and this is not accidental: the surrounding language/library surface fights such types, since there is no notion of relocation in the language itself. For instance, a relocate-only type cannot be returned by value, cannot be [push_backed](#), cannot be passed by value. Existing practice has naturally conformed to the first framing because that is the shape the language already supports.

The language-evolution argument supports the second framing. Several types are *naturally* relocate-only, in that they have no well-behaved moved-from state. The clearest example is a never-null [unique_ptr](#): since "never-null" is part of the invariant, there is no valid representation for a moved-

from source, and the type is not move-constructible. Yet, a never-null `unique_ptr` can be relocated perfectly well, by copying the pointer value to the destination and ending the lifetime of the source (without running its destructor). Under the first framing this type cannot be trivially relocatable, because there is no move+destroy sequence to optimize. Under the second framing it can be. Making such types first-class would require additional work elsewhere in the language and library (enabling e.g. passing and returning these objects by value, enabling a relocating `push_back`, and so on). A relocation feature under the second framing at least opens that door, whereas the first framing closes it before the discussion can begin.

A direct consequence of this question concerns types whose copy or move operations are deleted. Under the first framing, such types cannot be trivially relocated: there is no move+destroy sequence to optimize. Under the second framing the mechanism is independent of moves, so in principle such types could be relocated; however the default must still be that they are not relocatable. When the author of a type deletes a copy or a move, we do not know the underlying reason for that choice: the type's semantics may depend on the stability of its address, or on some other invariant we cannot see from the outside. Therefore, in these cases, implicit trivial relocation must be disabled (lacking an explicit signal from the type's author). The mechanism by which such an opt-in is granted (allowing, for instance, the never-null `unique_ptr` above to be declared trivially relocatable despite its deleted moves) is the subject of Question 5 ([§4.2 Question 5: User control: traits, keywords, or compiler determination?](#)).

§ 4.2. Question 5: User control: traits, keywords, or compiler determination?

How does a user declare that a type is trivially relocatable? The options range from keywords or attributes, to user specialization of type traits, or some combination of the above.

Two assumptions narrow this question. First, compiler-only automatic determination is a non-starter: the compiler cannot always tell whether a type whose author wrote special member functions preserves its invariants under a byte-for-byte move, so any such decision must ultimately come from the type's author. Second, the Rule of Zero case is uncontroversial: a type that declares none of its special member functions, and whose subobjects are all trivially relocatable, should be trivially relocatable automatically.

The question that remains concerns types for which the author declares at least one special member function, and it has three intertwined aspects: how the author spells the opt-in; how the opt-in composes through wrappers; and how the opt-in interacts with the existing rule-of-five machinery.

§ 4.2.1. Opt-in syntax

The opt-in can be spelled as an attribute, as a keyword (possibly context-sensitive), or as a user-specialized type trait. The choice is not purely a matter of taste: it depends on the answer to Question 4 (§4.1 Question 4: Optimization of move+destroy, or its own primitive?). If trivial relocation is an optimization of the move-plus-destroy sequence, then the program's observable semantics are unchanged and an attribute is an appropriate vehicle. Attributes in C++ are conventionally not allowed to change program semantics, and an optimization hint fits this convention.

Instead, if trivial relocation is a new primitive operation, then opting in changes what operations a type supports, which is a semantic change, and an attribute is no longer appropriate: a keyword (or a dedicated grammar production) is required.

A user-specializable type trait is a third possibility that sits uneasily between the two. It can carry semantics but at the cost of an awkward spelling (specializing a library template). Specializing a library template in order to describe a property of a type is not novel in the language: for instance, the tuple protocol used by structured bindings also uses a specialization to describe type properties.

§ 4.2.2. Conditional opt-in

Whatever the spelling, the opt-in must support being *conditional*: a class template `W` that wraps a `T` typically wants to be trivially relocatable exactly when `T` is. This case is pervasive in the Standard Library and in user code (`std::optional<T>`, `std::pair<T, U>`, `std::tuple<Ts...>`); all sufficiently generic wrapper types need to propagate the trait from their contents.

An opt-in that can only be written unconditionally forces some poor designs: either different specializations, or workarounds such as having a (0-sized) subobject which is conditionally trivially relocatable.

The C++26 design indeed had an unconditional keyword, and the lack of a conditional opt-in attracted several national body comments; in the discussions that followed, there was clear agreement that a future design needs to treat conditional opt-in as a first-class concern rather than a late addition.

§ 4.2.3. Interaction with the Rule of Five

We consider the current rules for implicit generation of special member functions a sad story.

The canonical example: if a type declares its destructor, even as `~T() = default;`, its move operations are implicitly deleted, but its copy operations continue to be generated; the deprecation of this

last behavior (that is, of the generation itself; the copy operations are *not* themselves deprecated) has stood for over 15 years without being acted on.

Bringing trivial relocation into this picture raises several choices.

One position is that the trivial relocation rules should follow the existing rule-of-five machinery as it stands, even where it is awkward: a type with `~T() = default;` has its moves suppressed, falls back to copies (which are still generated), and is therefore movable in the sense of Question 4's ([§4.1 Question 4: Optimization of move+destroy, or its own primitive?](#)) first framing, and on that basis automatically trivially relocatable.

Another position is that trivial relocation should not inherit these quirks: declaring any one of the special members should require the author to be explicit about trivial relocation as well, in the same way that declaring a destructor "should" (under the deprecation) require declaring the move and copy operations. The choice here has downstream consequences for how many existing types become trivially relocatable by default, and for how much retrofitting the Standard Library itself needs.

The trivial relocation design that was merged in C++26 illustrates how fraught this choice is, as it introduced a significant departure from the pre-existing rules. In particular, instead of looking at the mere presence of *user-declared* operations (which is the status quo regarding the Rule of Five special member functions, as well as the definition of trivially copyable), it looked at whether a hypothetical construction from a prvalue would select a user-provided constructor (cf. the definition of *default-movable* in [\[P2786R13\]](#)).

Types that have templated constructors (which, by definition, are never considered copy/move constructors) are classified differently by the Rule of Five machinery, by the other language type traits (especially `is_trivially_copyable`), and by C++26's detection of trivial relocation. With that detection, it would have been possible to define types that are trivially copyable but not trivially relocatable, which sounds bizarre.

This is to say: the mechanics between the Rule of Five and trivial relocation are not merely an "implementation detail"; any proposal that touches this area should either take the opportunity to do some (long overdue) cleanups, or carefully justify whether they're extending the status quo or deviating from it, weighing pre-existing behaviors, teachability of the new behaviors (if any), support in compilers and code linters, and so on.

Finally, the interaction with the Rule of Five also has implications on whether detecting if a type is automatically trivially relocatable can be implemented outside of the core language, for instance by using C++26's reflection. Reflection can tell us whether a class has user-declared or user-provided special member functions; but it cannot answer the query "does construction from a prvalue select a non-

defaulted constructor?". Even the `declcall(expression)` overload resolution hook from [P2825R5] cannot be used to query constructors (as they are not addressable functions).

§ 4.3. Question 6: Eligibility when subobjects are not trivially relocatable

If the author of a type opts in to trivial relocation, but one or more of its subobjects are not themselves trivially relocatable, what happens? Does the opt-in succeed (the author is trusted) or does it fail (the subobjects must cooperate)?

This question was framed in the design of [P1144R12] and [P2786R13] as "sharp-knife" versus "dull-knife" semantics, and the two papers picked opposite answers. This question was raised repeatedly during the process of standardizing trivial relocation in the C++26 cycle, so we believe it is important to record it here.

The "sharp-knife" position is that the author's opt-in wins. Once a type is declared trivially relocatable, it is; the compiler takes the author at their word, regardless of what the subobjects report.

The advantages of this position are mostly practical: a type can be marked trivially relocatable without waiting for its dependencies (including types from the Standard Library and/or from other third-party code) to be marked first. This matters because the upgrade story for trivial relocation spans an entire ecosystem. Moreover, types that are composed of many implementation-detail subobjects (a C++17-style `std::optional` built out of several private base classes and storage helpers, for instance) can be marked trivially relocatable at the top level without marking every internal subobject.

The disadvantage is that a wrong opt-in is silent: the author who marks a type trivially relocatable when one of its subobjects genuinely is not (for instance, `std::string` on implementations where it uses pointers into itself) introduces undefined behavior that the compiler cannot detect.

The "dull-knife" position is that the author's opt-in must be consistent with the subobjects. A type whose subobjects are not all trivially relocatable is not trivially relocatable, and the author's opt-in is ignored or becomes ill-formed. C++26's trivial-relocation design took the "dull-knife" position: the opt-in was ignored when subobjects did not cooperate, and the keywords were aptly named `_if_eligible` to underline the fact that the property is granted only in certain cases.

This position is grounded in precedent: the language does not, in general, let a user impose operations on a type from the outside. For instance, one cannot make a non-copyable type copyable by merely wrapping it. For the same reason, one should not be able to make a non-trivially-relocatable type trivially relocatable by wrapping it. The safety benefit is significant: a wrong opt-in at any layer is caught rather than silently miscompiled. The cost is on the upgrade story: trivial relocation cannot be exploit-

ed in a type until all of its dependencies (including Standard Library types that have not yet been audited and marked) have opted in.

§ 5. The scope of a relocation proposal

These are practical scoping questions about what must be included in a relocation proposal for it to be acceptable.

§ 5.1. Question 7: Should relocation work on automatic variables?

Does trivial relocation need to work on objects with automatic storage duration, or only on objects with dynamic lifetime?

One position is that trivial relocation covers only objects with dynamic lifetime. This is the position taken by [\[P2786R13\]](#) during the C++26 standardization cycle. The entry point was the library function `std::trivially_relocate`, which mandated that the parameters were only referring to objects with dynamic lifetime. This corresponds to the main use case for trivial relocation, that is, the optimization of `std::vector`-style container operations (container elements have dynamic lifetime). Automatic variables are out of scope, and the feature is relatively simple to specify.

The other position is that trivial relocation also reaches variables with automatic lifetime. The scope chosen by [\[P2786R13\]](#) left relocate-only types in an awkward position: they could exist as a type category, but they could not be cleanly handled as function locals: for instance, they could not be passed into functions or returned from them. The relocation primitive (`std::trivially_relocate`) simply did not work for these objects. Extending the scope of a trivial relocation proposal to this position finishes the story: relocate-only types become first-class value types. A second, independent benefit follows, and holds even under the optimization-of-move+destroy framing of Question 4 ([§4.1 Question 4: Optimization of move+destroy, or its own primitive?](#)): relocation of objects with automatic lifetime would help eliminate the "moved-from but still alive" state, a long-standing source of bugs in modern C++.

Now, extending to automatic variables comes with genuinely new machinery, on three fronts. First, the trigger mechanism for relocation of automatic variables is not obvious. A library function of the shape of `std::trivially_relocate` can continue to cover dynamic-lifetime objects, but some distinct operation may be required for automatic variables. This could take the form of a language operator, or of a different library entry point. Whether one mechanism suffices, or whether multiple are needed, is an open research and design question.

Second, the extension requires some sort of live-flow analysis. After a variable has been relocated away, any further use of it must be diagnosed statically, because the object is dead. C++ has no analysis of this kind today. A moved-from object is still usable, in a valid-but-unspecified state. The problem is known to be tractable; Rust enforces a comparable constraint as a matter of course. The cost of introducing such an analysis into C++, however, is unknown.

Third, the extension has implications for the ABI. Relocating an automatic variable into a function argument, or returning one by value from a function, raises questions about how the transfer is performed across frame boundaries. Whether existing ABI contracts can accommodate this operation, or whether a new one is required, is a question any concrete proposal will need to address.

Finally, this question interacts with Question 8 ([§ 5.2 Question 8: Must non-trivial relocation be included?](#)), which asks a related question along a different axis: trivial versus non-trivial. The two share a dimension, dynamic-only versus dynamic-plus-automatic, but they are not the same question. A design for trivial relocation that extends to automatic variables will be expected to accommodate non-trivial relocation too when that is addressed, because the live-flow machinery that lands for the one will be expected to carry the other. That interaction is discussed under Question 8.

§ 5.2. Question 8: Must non-trivial relocation be included?

May a trivial relocation proposal leave the general (non-trivial) relocation problem to a separate future feature, or must the two be designed together from the start?

Question 4 ([§ 4.1 Question 4: Optimization of move+destroy, or its own primitive?](#)) noted that attempts to add a general relocation primitive to C++ (see [\[P2839R0\]](#) and [\[P2785R3\]](#)) have stalled, and that the trivial case can in principle be specified on its own. The question here is whether a proposal that addresses only trivial relocation, leaving the general problem unsolved, is acceptable, or whether the two must be designed together from the start.

The case for a trivial-only proposal is that the trivial problem is substantially narrower than the general one. In the trivially relocatable case, a relocation reduces to a byte copy (possibly with a Question 2 ([§ 3.2 Question 2: Must trivial relocation always be bitwise?](#))-style fixup) followed by suppression of the source's destructor, for a subset of types that admit this treatment. On the other hand, designing a general relocation abstraction is a much larger undertaking. The benefits of the trivial case are concrete and large on their own, in particular for vector-like containers and for algorithms that rearrange elements. Pursuing trivial relocation without waiting on the general feature lets the standard realise those benefits on a shorter horizon.

Two considerations weigh against a trivial-only proposal. The first is a very minor point of consistency with existing nomenclature. Trivial properties in C++ today always describe a more general operation as trivial (however, see [\[P3279R0\]](#) for the "gritty" details): trivial default construction is a special case of default construction, trivial copy construction is a special case of copy construction, trivial destruction is a special case of destruction, and so on. A language that defines types as "trivially relocatable" without exposing any primitive notion of relocation would be the first trivial property without a corresponding general operation.

The second, more substantive concern is forward compatibility. Whatever syntax and mechanics a trivial-only proposal introduces (type-level properties, library functions, new core language operations, ...), they will need to interact with a future general relocation feature. A design calibrated only for the trivial case risks being too aggressive: it may foreclose design choices that a general relocation feature would otherwise have available, or force that feature to adopt a more complicated combined design than it would need on its own. For this reason, a concrete trivial-only proposal will likely be asked to show that its design does not close off plausible directions for a future general relocation feature, even if that feature is left to a separate proposal.

§ 5.3. Question 9: Must a trivial relocation proposal provide complete library support?

A library component of trivial relocation has several dimensions. Must a trivial relocation proposal address all of them, or may some be deferred to future additions?

The library side of trivial relocation has several dimensions. Each surfaced during the C++26 standardization cycle, and each was considered, sometimes partially deferred, before the feature was removed from the Working Draft. Any concrete proposal for C++29 will need to re-adjudicate them. The common question is whether each of these is gating for the proposal, or whether it may land as a follow-up.

The first question concerns the strength of the library's commitment to specific types. Any concrete proposal is expected to retain a baseline blanket authorization of the kind [\[P2786R13\]](#) provided: implementations are permitted to declare any suitable type trivially relocatable, as a quality-of-implementation matter. That blanket, taken on its own, has no teeth. Whether `std::unique_ptr<T>` or `std::tuple<int>` is trivially relocatable on a given implementation is simply a quality-of-implementation question, not a portable guarantee. The question is whether the proposal must, on top of the blanket, commit to named guarantees for specific standard types, so that users can rely on their trivial relocatability portably. Committing to such a list requires auditing major standard library implementations to establish which types can safely carry the guarantee. The choice applies uniformly to leaf vocabulary types (`std::unique_ptr`, `std::shared_ptr`, `std::string`, `std::vector`), com-

pound vocabulary types (`std::pair`, `std::tuple`, `std::optional`, `std::variant`, `std::expected`), and type-erased wrappers (`std::function`, `std::any`, `std::move_only_function`), whose trivial relocatability depends on small-buffer optimization decisions that differ across implementations.

The second question is whether container operations that rearrange elements in the middle, notably `insert` and `erase` on `std::vector`-like containers, must switch from move-assignment to relocation in order for the user-visible benefit of trivial relocation to materialize. [P4102R0] takes up that redesign deliberately as a separate proposal, untangling it from trivial relocation. A trivial relocation proposal may adopt the same approach and ship without this work, or may bundle the two.

The third question is allocator support. Trivial relocation bypasses the allocator, which is a problem for allocator-aware containers whose allocator tracks its elements and therefore needs to know when they move. [P3585R0] proposes an extension to `allocator_traits` to address exactly this interaction. A trivial relocation proposal may or may not include that extension. Without it, allocator-aware containers are restricted to allocators that do not need this knowledge, such as `std::allocator` and `std::pmr::polymorphic_allocator`.

The fourth question is whether the container requirements, as currently specified, are extended to accommodate relocate-only types. A container of a relocate-only type, such as a non-null form of `std::unique_ptr`, pushes against requirements that today assume at least moveability. [P4102R0] takes up this extension as well. Whether a trivial relocation proposal must land that extension before shipping, or whether it may defer, is a separate question from the preceding container one.

The fifth question is the algorithm side. It splits in two. Should standard algorithms such as `std::swap` and the move-based algorithms be permitted to use trivial relocation as an implementation strategy? And should their requirements be relaxed so that they apply to sequences of relocate-only types, enabling, for example, `std::sort` on a range of relocate-only objects? These are related but distinct axes, and a proposal may address one, the other, both, or neither.

One piece of library work already proceeds on this decoupling pattern: [P3516R2] proposes a family of `std::uninitialized_relocate` algorithms that work today via move-construct-and-destroy and can be extended to use trivial relocation once the language feature lands. It illustrates how the library surface can advance independently of the language primitive.

§ 6. Summary of design questions

Question	Positions and axes
----------	--------------------

Q1. Mechanism: byte copying or dedicated operations? (§ 3.1 Question 1: Mechanism: byte copying or dedicated operations?)	Bless byte-copying facilities (<code>memcpy</code> , <code>realloc</code> , and the like), or introduce a dedicated abstract-machine operation for trivial relocation.
Q2. Must trivial relocation always be bitwise? (§ 3.2 Question 2: Must trivial relocation always be bitwise?)	Always a pure bitwise copy (polymorphic types may not be trivially relocatable on architectures like arm64e), or allow a non-bitwise fixup step (mandatory entry point, or trait-dispatched).
Q3. Relocation and constant evaluation (§ 3.3 Question 3: Relocation and constant evaluation)	<code>constexpr</code> support from day one; never; or targeted but deferrable to a later cycle.
Q4. Optimization of move+destroy, or its own primitive? (§ 4.1 Question 4: Optimization of move+destroy, or its own primitive?)	Trivial relocation is the optimization of a move followed by a destroy, or its own primitive operation. The second framing admits relocate-only types (e.g. a never-null <code>unique_ptr</code>).
Q5. User control: traits, keywords, or compiler determination? (§ 4.2 Question 5: User control: traits, keywords, or compiler determination?)	Spelling of the opt-in (attribute, keyword, or specializable trait); support for conditional opt-in; interaction with the Rule of Five.
Q6. Eligibility when subobjects are not trivially relocatable (§ 4.3 Question 6: Eligibility when subobjects are not trivially relocatable)	"Sharp-knife": the author's opt-in wins. "Dull-knife": the opt-in must be consistent with the subobjects (C++26's choice).
Q7. Should relocation work on automatic variables? (§ 5.1 Question 7: Should relocation work on automatic variables?)	Dynamic storage duration and dynamic lifetime only (C++26's choice), or also automatic storage. The latter requires live-flow analysis and may have ABI implications, but makes relocate-only types first-class value types.
Q8. Must non-trivial relocation be included? (§ 5.2 Question 8: Must non-trivial relocation be included?)	Trivial-only is acceptable, or the general relocation feature must be designed alongside. Forward-compatibility considerations apply in either direction.

Q9. Must a trivial relocation proposal provide complete library support? (§ 5.3 [Question 9: Must a trivial relocation proposal provide complete library support?](#))

Five dimensions, each gating or deferrable: named-type guarantees vs. blanket authorization; container insert/erase redesign; allocator support; containers of relocate-only types; algorithm support.

§ 7. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Non-Normative References

[P1144R11]

Arthur O'Dwyer. *std::is_trivially_relocatable*. 15 May 2024. URL: <https://wg21.link/p1144r11>

[P1144R12]

Arthur O'Dwyer. *std::is_trivially_relocatable*. 15 October 2024. URL: <https://wg21.link/p1144r12>

[P2785R3]

Sébastien Bini, Ed Catmur. *Relocating prvalues*. 14 June 2023. URL: <https://wg21.link/p2785r3>

[P2786R13]

Pablo Halpern, Joshua Berne, Corentin Jabot, Pablo Halpern, Lori Hughes. *Trivial Relocatability For C++26*. 14 February 2025. URL: <https://wg21.link/p2786r13>

[P2825R5]

Gašper Ažman. *Overload resolution hook: declcall(unevaluated-call-expression)*. 17 March 2025. URL: <https://wg21.link/p2825r5>

[P2839R0]

Brian Bi, Joshua Berne. *Nontrivial relocation via a new "owning reference" type*. 15 May 2023. URL: <https://wg21.link/p2839r0>

[P2959R0]

Alisdair Meredith. *Container Relocation*. 15 October 2023. URL: <https://wg21.link/p2959r0>

[P3055R1]

Arthur O'Dwyer. *Relax wording to permit relocation optimizations in the STL*. 12 February 2024. URL: <https://wg21.link/p3055r1>

[P3233R0]

Giuseppe D'Angelo. *Issues with P2786 (Trivial Relocatability For C++26)*. 16 April 2024. URL: <https://wg21.link/p3233r0>

[P3279R0]

Arthur O'Dwyer. *CWG2463: What 'trivially fooable' should mean*. 15 May 2024. URL: <https://wg21.link/p3279r0>

[P3516R2]

Louis Dionne, Giuseppe D'Angelo. *Uninitialized algorithms for relocation*. 16 May 2025. URL: <https://wg21.link/p3516r2>

[P3559R0]

Arthur O'Dwyer. *Trivial relocation: One trait or two?*. 8 January 2025. URL: <https://wg21.link/p3559r0>

[P3585R0]

Pablo Halpern. *allocator_traits::is_internally_relocatable*. 13 January 2025. URL: <https://wg21.link/p3585r0>

[P3751R1]

Oliver Hunt, John Mccall. *A gentle introduction to pointer authentication*. 4 November 2025. URL: <https://wg21.link/p3751r1>

[P3780R0]

Giuseppe D'Angelo. *Detecting bitwise trivially relocatable types*. 10 July 2025. URL: <https://wg21.link/p3780r0>

[P3858R0]

David Sankel, Jon Bauman, Pablo Halpern. *A Lifetime-Management Primitive for Trivially Relocatable Types*. 6 October 2025. URL: <https://wg21.link/p3858r0>

[P3937R0]

Mingxin Wang, Zhihao Yuan. *Type Erasure Requirements For Future Trivial Relocation Design*. 15 December 2025. URL: <https://wg21.link/p3937r0>